

A Project Report

A Project Report
On
PARKING RECORD SYSTEM
Submitted in partial fulfillment of the requirement of
PROJECT-I
Of
Bachelor of Computer Engineering
Submitted to



Purbanchal University
Biratnagar , Nepal

Submitted By
Bigyata Pradhan(KCC03242024)
Nishita Dev(KCC02312024)
Sujan Pandey(KCC03322024)

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

30 May 2026

A Project Report
On
PARKING RECORD SYSTEM
Submitted in partial fulfillment of the requirement of
PROJECT-I
Of
Bachelor of Computer Engineering

Submitted to
Purbanchal University
Biratnagar, Nepal

Submitted By
Bigyata Pradhan(KCC03242024)
Nishita Dev(KCC02312024)
Sujan Panday(KCC03322024)

Project Supervisor
Er. Ronak Singh Karki

KANTIPUR CITY COLLEGE
Putalisadak, Kathmandu
30 May 2026

TABLE OF CONTENT

1.Abstract.....	i
2.Acknowledgement.....	ii
3.Preface.....	iii
4.Declaration.....	iv
5.Supervisor's approval.....	v
6.Certificate from Department.....	vi
7.Introduction.....	1
7.1.Overview.....	2
7.2.Problem Statement.....	3
7.3.Objective.....	4
7.4.Features.....	5
7.5.Significance.....	6
7.6.Scope and limitations.....	6
8. Literature Review.....	7
8.1.ParkingPass.com Overview.....	7
8.1.1.ParkingPass Pros.....	7
8.1.2.ParkingPass Cons.....	8
8.2.SpotHero Overview.....	8
8.2.1.SpotHero Pros.....	8
8.2.2.SpotHero Cons.....	8
8.3.T2 Systems Overview.....	9
8.3.1. T2 Systems Pros.....	9
8.3.2.T2 Systems Cons.....	9
9.Methodology.....	10
9.1.Software Development Life Cycle.....	10

9.2.Technology and Tools used.....	11
9.2.1.Programming language.....	11
9.2.2.Standard Library Used.....	11
9.2.3.Specified Language Used.....	12
9.3 Assignment of Roles and Responsibility.....	12
10.System Analysis.....	13
10.1.Requirement Analysis.....	13
10.2.Technical Feasibility.....	13
10.3.Economic Feasibility.....	14
11.System Design.....	15
11.1.Algorithm.....	15
11.2.Flowchart.....	18
12.System Architecture.....	41
12.1.Data Flow Diagram.....	41
12.1.1.Level 1 Data Flow Diagram.....	41
12.1.2.Level 2 Data flow Diagram.....	42
12.2.Object Oriented.....	43
12.2.1.Use case diagram.....	43
12.2.2.Sequence diagram.....	44
13.System Development and Implementation.....	45
13.1System Development.....	45
13.2 System Implementation.....	46
14.Tools and Testing.....	47
14.1.Tools used in testing.....	47
14.2.Test Cases.....	48
15.Conclusion.....	50

List of Figure

Figure 1: Prototype Model Diagram

Figure 2: Flowchart

Figure 3: DFD Level 0

Figure 4: DFD Level 1

Figure 5: Use Case Diagram

Figure 6: Sequence Diagram

Abstract

Abstract

Our aim is to design and create a parking record system. The Parking Record System automates the tracking of vehicle entry, exit, and parking duration within a parking facility.

It stores vehicle details, allocates available spaces, displays parking charge/fee, maintains real-time records to and improves operational efficiency. The system supports secure data management, provides insights into space utilization, and generates reports for administrative use.

Overall, This system offers a scalable and efficient solution suitable for commercial, institutional, and public parking facilities.

Acknowledgements

We would like to express our gratitude and thankfulness to respect Er. Ronak Singh Karki, our project supervisor, who gave us his time with valuable guidance ,constant encouragement and suggestions throughout the project's planning, designing and report writing.

We would also like to thank Mr. Subash Rajkarnikar, Head of Director of the computer and electronic department, Rubim Shrestha, coordinator, department of IT and Engineering for their guidance and support. We would also like to express my gratitude towards our principal Mr. Raju Kattel for giving us this great opportunity to do a project on 'Parking Record System' . Without their support and suggestions, this project would not have been completed.

Lastly , we would like to express our sincere thanks to all our friends, teachers and others who helped us directly or indirectly during this project.

Bigyata Pradhan(KCC032420240)

Nishita Dev(KCC02312024)

Sujan Panday(KCC03322024)

PREFACE

The **Parking Record Keeping System** is designed to manage and organize vehicle parking information efficiently. The purpose of this project is to understand the process of designing and implementing a system that can record, store, and manage parking details effectively.

The project focuses on simplifying the traditional manual parking record system by introducing a computerized approach. It helps in maintaining accurate records of vehicles, including entry and exit time, vehicle number, and other necessary details.

DECLARATION

We hereby declare that this project report titled "**PARKING RECORD KEEPING SYSTEM**" submitted in partial fulfillment of the "Bachelor's in Computer Engineering" is a record of our original work completed under Er. Ronak Singh Karki supervision has not served as the foundation for the award of any other degree or diploma, in this institution or university or any other. Everywhere the findings of others have been quoted, suitable acknowledgments have been made in accordance with the ethical practice of reporting scientific information.

Bigyata Pradhan(KCC032420240)

Nishita Dev(KCC02312024)

Sujan Panday(KCC03322024)

Date:

SUPERVISOR'S APPROVAL

This is to certify that the major project entitled “**Parking Record keeping System**” undertaken and demonstrated by Bigyata Pradhan(KCC03242024), Nishita Dev(KCC02312024), Sujan Panday(KCC03322024) has been successfully completed under my supervision as a partial fulfillment of the requirements for the degree of bachelor's in computer engineering III under Purbanchal University. I henceforth approve of this project to be awarded the certificate by the concerned authority. During supervision, I found students hardworking, skilled, and ready to undertake any professional work related to this field in future

Er. Ronak Singh Karki

Department of Computer Engineering

Kantipur City College

(Project Supervisor)

Date: 30th November 2023

CERTIFICATE FROM DEPARTMENT

Following the Supervisor's Approval and Examiner's Acceptance, the project entitled "**Parking record keeping system**" submitted by Bigyata pradhan (03242024), Nishita dev (02312024), Sujan panday (03322024) as a partial fulfillment of the requirement for the degree of Bachelor's in Computer Engineering III under Purbanchal University, has been officially awarded by this certificate.

I wish the students all the best for their future endeavors

Er. Subash Rajkarnikar
HOD, Department of Computer Engineering
Kantipur City College

Introduction

CHAPTER-7

7. INTRODUCTION

The Parking Record Keeping System is a comprehensive console-based software designed to efficiently manage parking operations for both two-wheelers and four-wheelers. In modern urban areas, managing parking spaces can be a challenging task due to the high volume of vehicles and the need for accurate billing and record-keeping. This system addresses these challenges by providing a structured and automated solution for parking lot management.

With this system, parking attendants can easily record vehicle entries and exits, calculate parking fees based on the duration of stay, and generate receipts for customers. It also provides secure user authentication, ensuring that only authorized personnel can access and operate the system. To enhance security, the system allows a maximum of three login attempts before denying access.

All vehicle data is stored in files, enabling long-term tracking and quick retrieval of records. Users can register, login , enter , exit and search records of vehicles . Additionally, a search feature allows the retrieval of a specific vehicle's record by entering its vehicle number.

The system also calculates and displays the total daily income, helping parking lot managers monitor revenue efficiently. Designed to run entirely in the console, it offers a simple, interactive, and user-friendly interface that is easy to operate even for non-technical users

7.1 Overview

The Parking Record Keeping System is a console-based application developed to automate and simplify the management of vehicle parking operations. It is designed for small to medium parking lots handling both two-wheelers and four-wheelers, providing an organized way to track vehicles, generate receipts, and calculate parking fees.

The system starts with secure user authentication, allowing only registered users to access its functions. It supports a maximum of three login attempts to enhance security. Once logged in, users can perform a variety of operations, including vehicle entry and exit, where entry and exit times are recorded, and parking fees are automatically calculated based on the duration of stay and vehicle type.

A key feature of the system is comprehensive record management. All vehicle details—such as vehicle number, type, entry/exit times, status, and receipt numbers—are stored in files for future reference. Users can view all records, including a detailed summary of total vehicles, types, vehicles currently parked, exited vehicles, and daily income. Additionally, the search functionality allows quick retrieval of specific vehicle records by entering the vehicle number.

The system provides center-aligned console output, making it user-friendly and easy to navigate. With features like automated billing, daily income calculation, and vehicle summaries, it enhances the efficiency of parking lot management.

Overall, the system offers a simple, reliable, and practical solution for managing parking spaces effectively while maintaining accurate records and revenue tracking.

7.2 Problem Statement

In today's fast-growing urban environments, parking management has become a significant challenge. Parking lots often face issues such as inefficient tracking of vehicles, manual errors in recording entry and exit times, delayed billing, and difficulty in monitoring parking lot occupancy. These problems can lead to revenue loss, customer dissatisfaction, and operational inefficiencies.

Traditional manual methods of managing parking records, such as handwritten logs, are time-consuming, prone to mistakes. Additionally, generating reports for daily income, tracking the number of vehicles, or searching for specific vehicle records is difficult and inefficient without an automated system.

The Parking Record Keeping System is designed to address these problems by providing a secure, reliable, and automated solution. It enables real-time tracking of vehicle entry and exit, automatic fee calculation, and maintains accurate records for all vehicles. By automating these processes, the system ensures efficiency, transparency, and convenience for both parking lot operators and vehicle owners.

7.3 Objective

The major objective of the project are as follows;

- To automate vehicle entry and exit in the parking lot.
- To calculate parking fees automatically based on vehicle type and duration.
- To maintain accurate records of all vehicles and their status.
- To allow users to search for specific vehicle records quickly.
- To provide a summary of total vehicles,types and daily income.

4.4 Features

The major features of the project:

1. Vehicle Registration

- Vehicles must be registered before they can enter the parking lot.
- Stores Vehicle Number and Vehicle Type (2-wheeler or 4-wheeler) in user.txt .

2. Vehicle Entry

- Allows only registered vehicles to enter.
- Records Entry Time (hour and minute) and vehicle status as "IN".
- Stores entry data in parking.txt.

3. Vehicle Exit

- Prompts for Exit Time for vehicles with "IN" status.
- Calculates duration of parking using entry and exit times.
- Updates vehicle status to "OUT" in the file.
- Displays Vehicle Number, Type, Entry Time, Exit Time, Duration, and Fee.

4. Fee Calculation

- Fee depends on vehicle type:
 - 2-Wheelers → Rs. 10 per hour
 - 4-Wheelers → Rs. 20 per hour
- Partial hours are rounded up to the next hour.

5. File-Based Storage

- Persistent storage using files: user.txt and parking.txt.
- Can handle multiple vehicles across program runs.
- Ensures records are updated after entry and exit.

6. Menu-Driven Interface

- User-friendly menu to choose:
 - Register Vehicle
 - Vehicle Entry
 - Vehicle Exit
 - Exit Program

7. Input Validation

- Check if the vehicle is registered before entry.
- Prevents exit of unregistered or already exited vehicles.

8. User-Friendly Messages

- Prints success messages after registration, entry, and exit.
- Prints THANK YOU message when the program exits.

9. Supports Multiple Vehicle Types

- Differentiates between 2-wheelers and 4-wheelers.
- Calculates fees accordingly.

10. Total Income Report

- Calculate total money earned
- Count:
 - Total 2-wheelers
 - Total 4-wheelers
 - Total vehicles exited

11. Parking Slot Management

- Limit total slots (eg, 50)
- Prevent entry if full
- Track available slots

12. Duplication

- A vehicle can only be parked once at a time.
- Re-entry is allowed only after exit, ensuring data consistency and proper system control.

7.5 Significance

A parking record system is important for managing vehicle parking in an organized and systematic way. It helps to maintain accurate records of vehicle entry and exit, reducing manual errors and saving time. The system improves efficiency by calculating parking duration and fees automatically. It also enhances security by keeping vehicle details stored in records, which can be useful for tracing and verification purposes. It maintains detailed and organized records of all vehicles, their status, and transactions. It allows instant searching of vehicle records and viewing summaries of occupancy and daily income. It helps parking lot managers monitor and track daily income effectively.

7.6 Scope and limitation

Scope

- The system is designed for small to medium parking lots handling two-wheelers and four-wheelers.
- Automates the vehicle entry and exit process, including recording times and calculating parking fees.
- Maintains detailed records for all vehicles, including their status (IN/OUT), type, and receipt number.
- Provides search functionality for quick retrieval of specific vehicle records.
- Generates summaries like total vehicles, occupancy status, and daily income for better management.
- Offers a simple console-based interface, making it easy to use and deploy on basic systems without advanced hardware.

Limitations:

- The system cannot handle more than the defined parking slots (10 two-wheelers, 10 four-wheelers) at a time.
- It is console-based, so it has a simple text interface without graphical

elements.

- The program does not support network or multi-user access; only one user can operate at a time.
- Data is stored in text files, so it is not suitable for very large-scale parking systems requiring databases.
- Time input and fee calculation rely on manual entry of HH MM, so incorrect entries may cause errors.

CHAPTER-8

8. LITERATURE REVIEW

8.1 ParkingPass.com

Offers a paperless parking management program for properties, featuring online registration, integrated license plate recognition (LPR), guest management, and detailed reporting tools. This system simplifies tasks like issuing permits, monitoring parking spaces, and enforcing policies, making it easy for property managers to keep track of everything. This system boosts operational efficiency and increases customer satisfaction by providing a smoother and more controlled parking experience, making it essential for modern parking management.

8.1.1 Pros

- Streamlined & Paperless Parking Management.
- Efficient Permit and Vehicle Tracking.
- Built-In Revenue Tools.
- Enhanced Reporting & Insights.
- Customization.

8.1.2 Cons

- Cost Can Be High.
- Internet Dependency.
- Hardware Requirements.
- Limited Offline Functionality.
- Not Ideal for Very Small Parking Areas.

8.2 SpotHero

It is a leading digital parking marketplace in North America, letting drivers find, book, and pay for guaranteed parking spots in garages, lots, and valet services via its app or website, offering prepaid passes for hourly, daily, monthly, or event parking, saving time and stress by eliminating guesswork, especially for airports, concerts, and major city events.

8.2.1 Pros

- Wide City Coverage.
- Real-Time Availability.
- Clear Directions & Access Info.
- Flexible Parking Options.
- Real-Time Availability.

8.2.2 Cons

- No Guarantee for All Situations.
- Less Control for Parking Owners.
- Internet & App Dependency.
- Refund & Cancellation Restrictions.

8.3 T2 Systems

T2 Systems provides a comprehensive, integrated platform for managing all aspects of parking, from access and revenue to enforcement, using smart technology like License Plate Recognition (LPR) and mobile apps for universities, cities, and private operators. Their solutions offer real-time control, contactless payments, automated permits, efficient citation management, and data-driven insights, creating streamlined, user-friendly parking experiences with solutions like T2 Flex, UP Safety, and T2 Velocity.

8.3.1 Pros

- Ideal for Large Organizations.
- Advanced Reporting & Analytics.
- Reliable & Industry-Trusted.
- Cloud-Based Access.
- Comprehensive Parking Management.

8.3.2 Cons

- App Limitations for Specific Use-Cases.
- Dependence on Vendor & Integration Limits.
- Complex Interface for New Users.
- Learning Curve & Training Needs.

CHAPTER -9

9.Methodology

9.1 Software Development Life Cycle

We selected a waterfall model to create this project since criteria were clear and we used simple information.

Prototype Model

The modified prototype model is a software development approach that provides the flexibility to implement changes and see their effects. To improve the preciseness of a new design and to reduce the risk of developing a software product that doesn't meet the need of end-users or stake holders while also reducing development time.

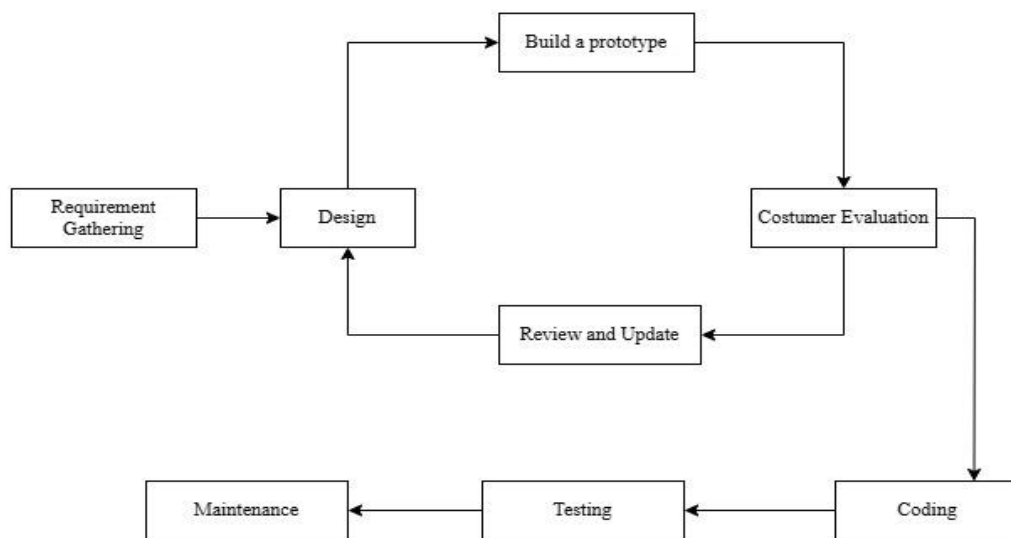


Figure 1: Prototype Model

9.2 TECHNOLOGY AND TOOLS USED

This Parking Record System is developed using C programming language with standard system libraries and file handling concepts. It is a console-based application designed to run on Windows environments.

9.2.1 Programming Language

- C Language
- Used for implementing the complete logic of the system
- Provides fast execution and low-level memory control

9.2.2 STANDARD LIBRARIES USED

The following header files are used in the project:

- `stdio.h`
Used for input and output operations like `printf()` and `scanf()`.
- `stdlib.h`
Used for general utility functions such as `exit()`.
- `string.h`
Used for string operations like `strcmp()` and `strcpy()`.
- `conio.h`
Used for console input/output functions like `getch()` and password masking.
- `windows.h`
Used for console manipulation functions like `gotoxy()` and cursor positioning.

9.2.3 SPECIFIED SOFTWARES USED

- Operating System: Windows 7 / 8 / 10 / 11
- Programming Language: C Language
- Compiler / IDE:
 - Dev C++
 - Turbo C++
 - Code::Blocks
- Text Editor (Optional): Notepad / Notepad++
- Execution Environment: Command Prompt / Console Window
- Libraries Supported By Compiler: Standard C libraries (`stdio.h`, `stdlib.h`, `string.h`, `conio.h`, `windows.h`)

9.3 Assignment of Roles and Responsibility

Member Name	Roles and Responsibilities	
Bigyata Pradhan		
Nishita Dev		
Sujan Pandey		

Chapter 10

10. System Analysis

10.1.Requirement Analysis

Requirement analysis is the initial and one of the most important phases in the

development of the Parking Record Keeping System In this phase, all the requirements of the system are identified, studied, and analyzed to ensure that the software fulfills the needs of users effectively. The system is designed to handle vehicle registration, vehicle entry and exit management, parking fee calculation, parking slot tracking, and report generation. The system requires a vehicle registration feature where vehicles must be registered before entering the parking area. During registration, the vehicle number and vehicle type, such as 2-wheeler or 4-wheeler, are stored permanently in the user.txt file. The system also includes a vehicle entry module that allows only registered vehicles to enter the parking area. When a vehicle enters, the system records the entry time and updates the vehicle status as “IN” in the parking.txt file. Similarly, the vehicle exit module records the exit time, calculates the parking duration, computes the parking fee according to the vehicle type, and updates the vehicle status to “OUT”.The parking fee calculation is another important requirement of the system.

10.2. Technical Feasibility

This activity is also known as scope study. Perform and evaluate feasibility studies like cost-benefit analysis, technical feasibility, time feasibility and operational feasibility for the project. Scheduling should be made using charts. A Feasibility study is being carried out to decide whether the proposed system is feasible for the company. It begins with a request from the user for a new system. Determine the feasibility for the new system .

10.3. Economic Feasibility

Economic feasibility means checking whether the system is cost-effective and affordable. The Parking Record Keeping System is economically feasible because it requires very low cost for development and maintenance. The software tools used for development are free, and the system can run on a normal computer without

expensive hardware. Since the system automates parking management, it reduces manual work, saves time, and minimizes human errors. Features like automatic fee calculation, parking record storage, and report generation improve efficiency and reduce paperwork .

Chapter 11

11. System Design

11.1 Algorithm

Step 1: Start Program

1. Start the system.
2. Display the main menu:
 - Register
 - Login
 - Exit

Step 2: Register User

1. Enter username.
2. Enter password.
3. Enter role (admin/user).
4. Open `users.txt` file in append mode.
5. Store user details in file.
6. Close file.
7. Display registration success message.

Step 3: Login User

1. Enter username.
2. Enter password.
3. Open `users.txt` file in read mode.
4. Compare entered credentials with stored records.
5. If username and password match:
 - Login successful.
 - Store current role.
6. Else:
 - Display invalid credentials message.
 - Reduce remaining attempts.
7. If attempts become zero:

- Exit program.

Step 4: Admin Operations

1. Display admin menu:
 - Vehicle Entry
 - Vehicle Exit
 - View Records
 - Search Record
 - View Daily Income
 - Logout
2. Read user choice.
3. Perform selected operation.
4. Repeat until logout.

Step 5: User Operations

1. Display user menu:
 - Vehicle Entry
 - Vehicle Exit
 - Search Record
 - Logout
2. Read user choice.
3. Perform selected operation.
4. Repeat until logout.

Step 6: Add Vehicle

1. Check parking slot availability.
2. If slots are full:
 - Display “No Slots Available”.
 - Return to menu.

3. Enter vehicle number.
4. Check duplicate vehicle:
 - If already inside:
 - Display duplicate messages.
 - Stop the process.
5. Enter vehicle type.
6. Enter entry time.
7. Set status as **IN**.
8. Generate receipt number.
9. Save record in **parking.txt**.
10. Update occupied slot count.
11. Display success message.

Step 7: Exit Vehicle

1. Enter vehicle number.
2. Open **parking.txt**.
3. Search vehicle record.
4. If vehicle not found:
 - Display error message.
5. If vehicle already exited:
 - Display an already exited message.
6. Enter exit time.
7. Calculate parking fee.

Parking fee calculation:

$$\text{Total Minutes} = (\text{Out Hour} \times 60 + \text{Out Minute}) - (\text{In Hour} \times 60 + \text{In Minute})$$

$$\text{Total Minutes} = (\text{Out Hour} \times 60 + \text{Out Minute}) - (\text{In Hour} \times 60 + \text{In Minute})$$

Then:

For

two-wheelers:

$$\text{Fee} = \text{Hours} \times 20 \quad \text{\text{Fee} = \text{Hours} \times 20}$$

For four-wheelers:

$$\text{Fee} = \text{Hours} \times 40 \quad \text{\text{Fee} = \text{Hours} \times 40}$$

8. Update vehicle status to **OUT**.
9. Update daily income.
10. Rewrite updated records into file.
11. Display parking fee.

Step 8: Search Vehicle

1. Enter vehicle number.
2. Open **parking.txt**.
3. Search matching record.
4. If found:
 - Display vehicle details.
5. Else:
 - Display “Record Not Found”.

Step 9: Display Records

1. Open **parking.txt**.
2. Read all parking records.
3. Display all vehicle information.
4. Close file.

Step 10: View Income

1. Read total daily income variable.
2. Display total income.

Step 11: End Program

1. Display thank you message.

2. Terminate program.

11.2. Flowchart Diagram

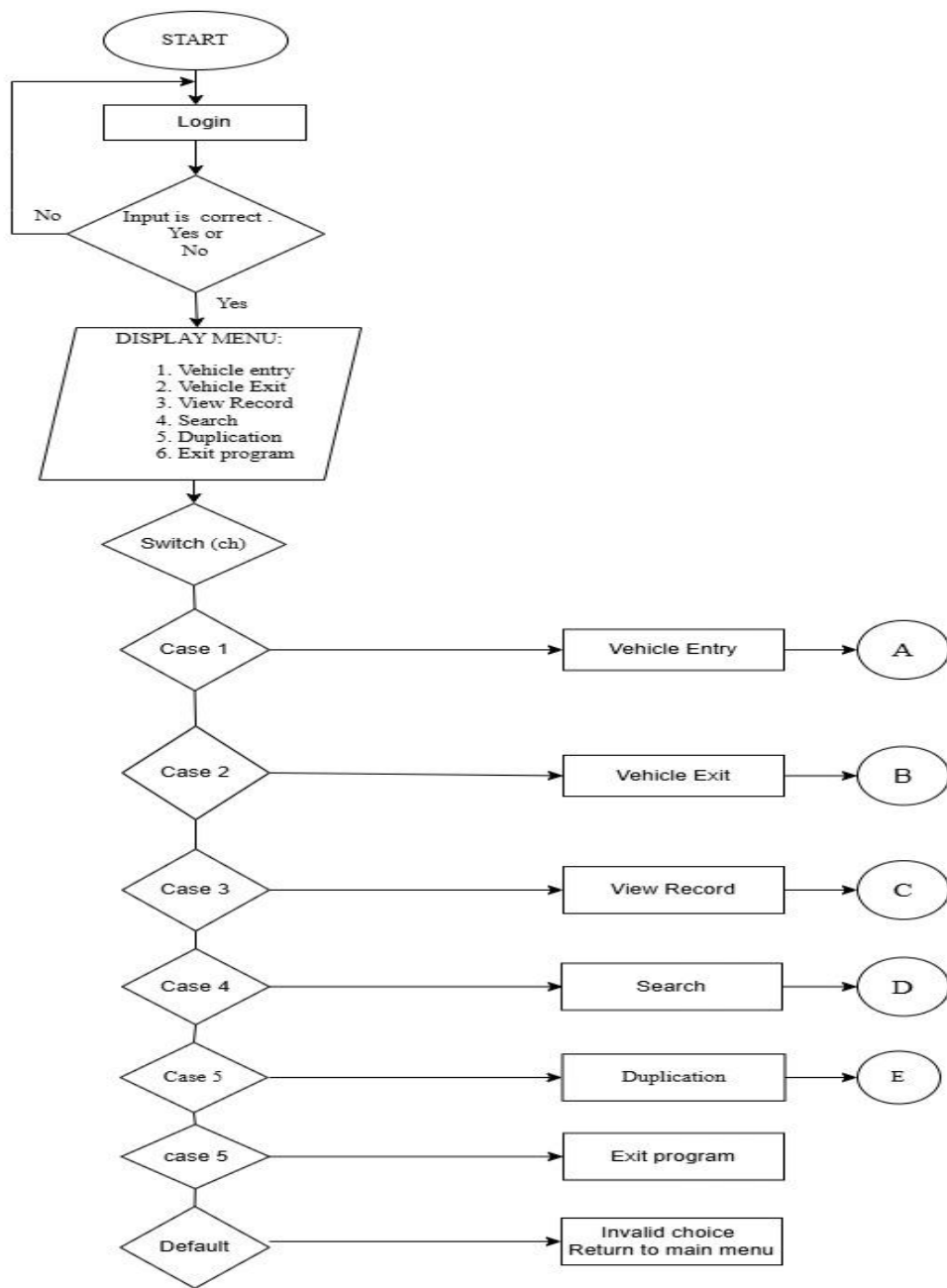


Figure:2

11.2.1. Vehicle Entry

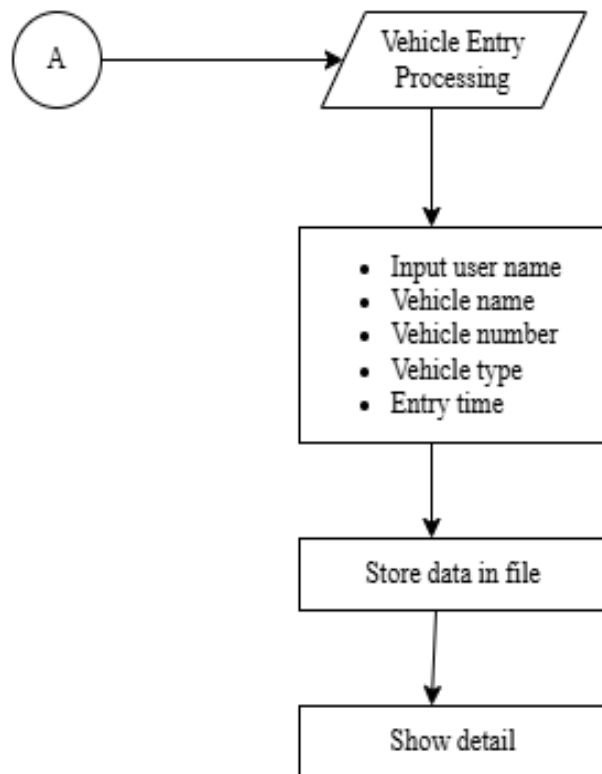


Figure:2.1

11.2.2. Vehicle Exit

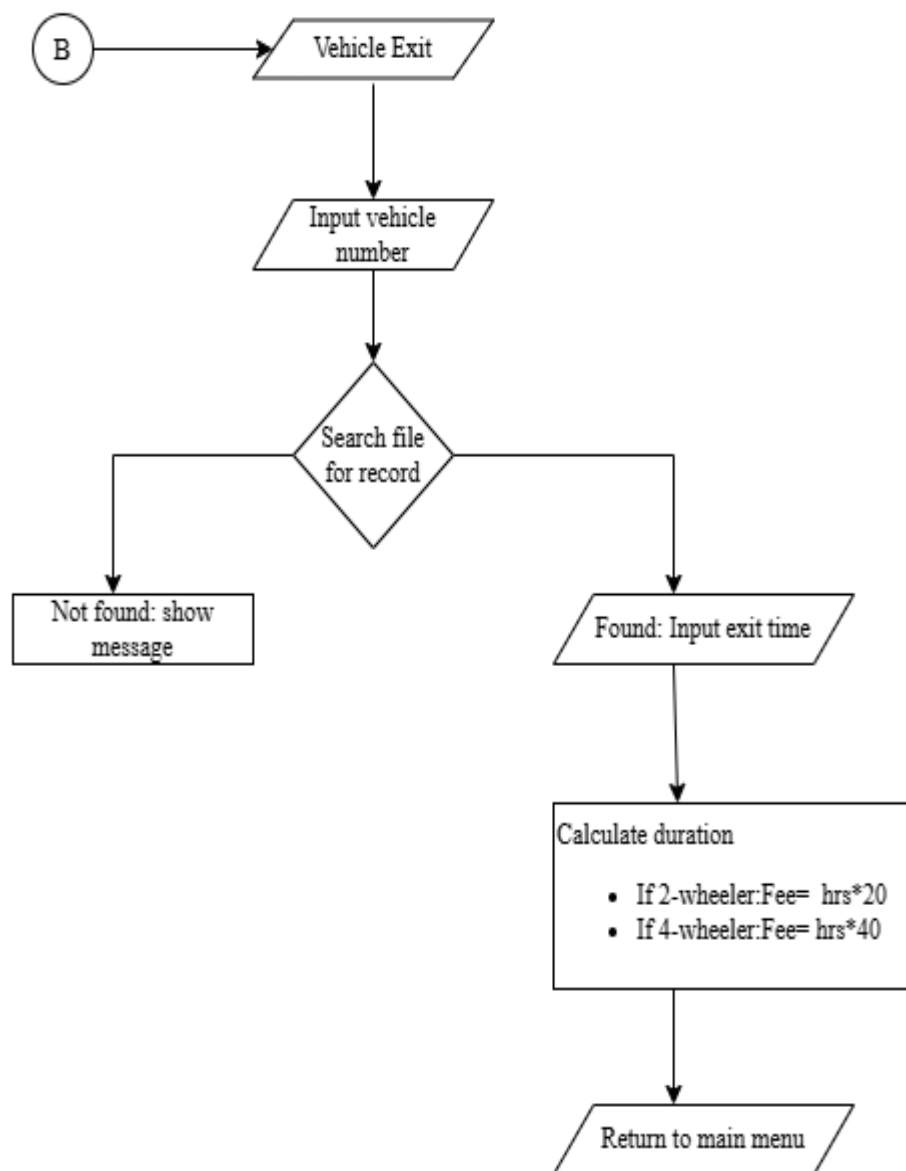


Figure:2.2

11.2.3. View Record

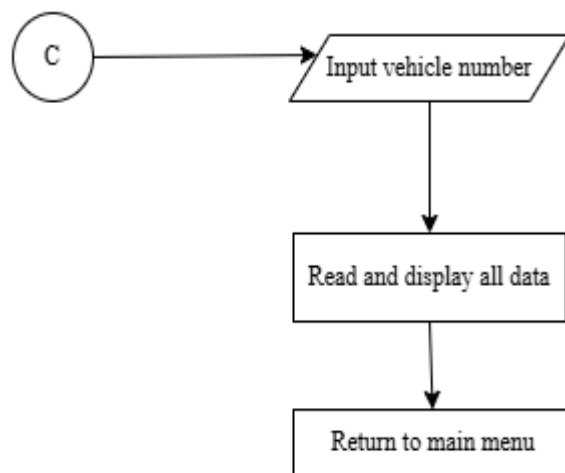


Figure:2.3

11.2.4. Search

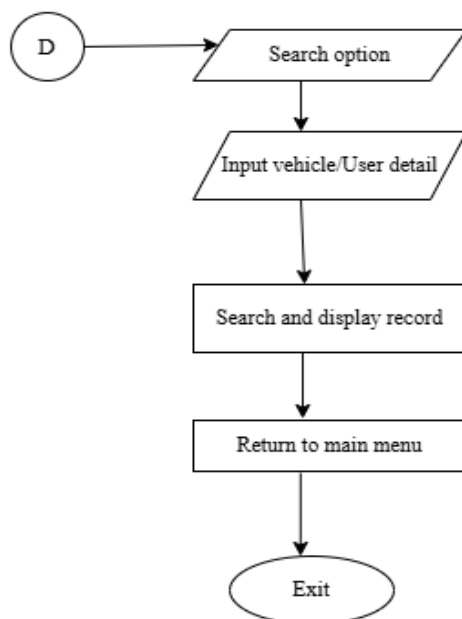


Figure:2.4

11.2.5. DUPLICATION

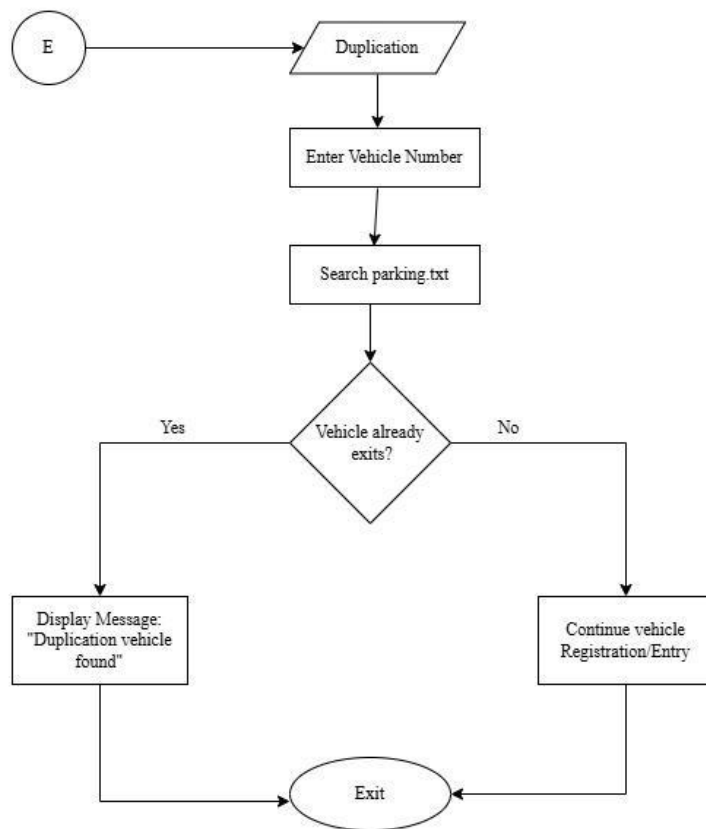


Figure:2.5

Coding

```

/* =====

PARKING RECORD SYSTEM

WITH ADMIN AND USER

===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>
#include <windows.h>

#define TWO_WHEELER_SLOTS 10
#define FOUR_WHEELER_SLOTS 10

/* ----- STRUCTURES ----- */

struct User
{
    char username[20];
    char password[20];
    char role[10];
};

struct Parking
{
    char vehicleNo[20];
    int type;

    int inH, inM;
    int outH, outM;

    char status[5];

```

```

    int receiptNo;
};

/* ----- GLOBAL VARIABLES ----- */

int occupied2W = 0;
int occupied4W = 0;

int dailyIncome = 0;

int receiptCounter = 1;

char currentRole[10];

/* ----- UTILITY FUNCTIONS ----- */

void gotoxy(int x, int y)
{
    COORD c;

    c.X = x;
    c.Y = y;

    SetConsoleCursorPosition(
        GetStdHandle(STD_OUTPUT_HANDLE), c);
}

/* ----- HIDDEN PASSWORD ----- */

void inputHiddenPassword(char pass[])
{

```



```

char ch;

int i = 0;

while (1)
{
    ch = getch();

    if (ch == 13)
        break;

    else if (ch == 8 && i > 0)
    {
        printf("\b \b");
        i--;
    }

    else if (i < 19)
    {
        pass[i++] = ch;
        printf("*");
    }
}

pass[i] = '\0';
}

/* ----- REGISTER USER ----- */

void registerUser()
{
    struct User u;

```

```

FILE *fp;

fp = fopen("users.txt", "a");

if(fp == NULL)
{
    gotoxy(30,10);
    printf("File Error!");

    getch();

    return;
}

system("cls");

gotoxy(30,4);
printf("***** USER REGISTRATION *****");

gotoxy(30,6);
printf("Enter Username: ");
scanf("%s", u.username);

gotoxy(30,7);
printf("Enter Password: ");
inputHiddenPassword(u.password);

gotoxy(30,8);
printf("Enter Role (admin/user): ");
scanf("%s", u.role);

```

```

    fprintf(fp,
        "%s %s %s\n",
        u.username,
        u.password,
        u.role);

fclose(fp);

gotoxy(30,10);
printf("User Registered Successfully!");

getch();
}

/* ----- LOGIN FUNCTION ----- */

int loginUser()
{
    struct User u;

    char uname[20];
    char pass[20];

    int attempts = 3;

    while(attempts > 0)
    {
        FILE *fp;

        fp = fopen("users.txt", "r");

        if(fp == NULL)

```

```

{
    gotoxy(30,10);
    printf("No Users Found!");

    getch();

    return 0;
}

system("cls");

gotoxy(30,4);
printf("***** LOGIN *****");

gotoxy(30,6);
printf("Enter Username: ");
scanf("%s", uname);

gotoxy(30,7);
printf("Enter Password: ");
inputHiddenPassword(pass);

int valid = 0;

while(fscanf(fp,
    "%s %s %s",
    u.username,
    u.password,
    u.role) != EOF)
{
    if(strcmp(u.username, uname) == 0 &&
        strcmp(u.password, pass) == 0)

```

```

    {
        valid = 1;

        strcpy(currentRole, u.role);

        break;
    }
}

fclose(fp);

if(valid)
{
    gotoxy(30,9);
    printf("Login Successful!");

    gotoxy(30,10);
    printf("Role: %s", currentRole);

    getch();

    return 1;
}

else
{
    attempts--;

    gotoxy(30,9);
    printf("Invalid Credentials!");

    gotoxy(30,10);

```

```

        printf("Attempts Left: %d", attempts);

        getch();
    }
}

gotoxy(30,12);
printf("Too Many Failed Attempts!");

getch();

exit(0);
}

/* ----- CALCULATE FEE ----- */

int calculateFee(int type,
                int inH,
                int inM,
                int outH,
                int outM)
{
    int totalInMinutes =
        inH * 60 + inM;

    int totalOutMinutes =
        outH * 60 + outM;

    int totalMinutes =
        totalOutMinutes - totalInMinutes;

    if(totalMinutes <= 0)

```

```

        return -1;

int hours = totalMinutes / 60;

if(totalMinutes % 60 != 0)
    hours++;

if(type == 2)
    return hours * 20;

else
    return hours * 40;
}

/* ----- DUPLICATE VEHICLE ----- */

int isDuplicateVehicle(char vno[])
{
    FILE *fp;

    struct Parking p;

    fp = fopen("parking.txt", "r");

    if(fp == NULL)
        return 0;

    while(fscanf(fp,
        "%s %d %d %d %d %d %s %d",
        p.vehicleNo,
        &p.type,
        &p.inH,

```

```

        &p.inM,
        &p.outH,
        &p.outM,
        p.status,
        &p.receiptNo) != EOF)
    {
        if(strcmp(p.vehicleNo, vno) == 0 &&
            strcmp(p.status, "IN") == 0)
        {
            fclose(fp);

            return 1;
        }
    }

    fclose(fp);

    return 0;
}

/* ----- VEHICLE ENTRY ----- */

void vehicleEntry()
{
    if(occupied2W >= TWO_WHEELER_SLOTS &&
        occupied4W >= FOUR_WHEELER_SLOTS)
    {
        gotoxy(30,8);
        printf("No Slots Available!");

        getch();
    }
}

```



```

        return;
    }

    struct Parking p;

    FILE *fp;

    fp = fopen("parking.txt", "a");

    system("cls");

    gotoxy(30,4);
    printf("***** VEHICLE ENTRY *****");

    gotoxy(30,6);
    printf("Enter Vehicle Number: ");
    scanf("%s", p.vehicleNo);

    if(isDuplicateVehicle(p.vehicleNo))
    {
        gotoxy(30,8);
        printf("Vehicle Already Inside!");

        fclose(fp);

        getch();

        return;
    }

    gotoxy(30,7);
    printf("Vehicle Type (2/4): ");

```

```

scanf("%d", &p.type);

gotoxy(30,8);
printf("Enter Entry Time (HH MM): ");
scanf("%d %d", &p.inH, &p.inM);

p.outH = 0;
p.outM = 0;

strcpy(p.status, "IN");

p.receiptNo = receiptCounter++;

fprintf(fp,
        "%s %d %d %d %d %d %s %d\n",
        p.vehicleNo,
        p.type,
        p.inH,
        p.inM,
        p.outH,
        p.outM,
        p.status,
        p.receiptNo);

fclose(fp);

if(p.type == 2)
    occupied2W++;

else
    occupied4W++;

```

```

gotoxy(30,10);
printf("Vehicle Entered Successfully!");

getch();
}

/* ----- VEHICLE EXIT ----- */

void vehicleExit()
{
    struct Parking p[100];

    FILE *fp;

    char vno[20];

    int i;
    int n = 0;

    int found = 0;

    system("cls");

    gotoxy(30,4);
    printf("***** VEHICLE EXIT *****");

    gotoxy(30,6);
    printf("Enter Vehicle Number: ");
    scanf("%s", vno);

    fp = fopen("parking.txt", "r");

```

```

if(fp == NULL)
{
    gotoxy(30,8);
    printf("No Records Found!");

    getch();

    return;
}

while(fscanf(fp,
    "%s %d %d %d %d %d %s %d",
    p[n].vehicleNo,
    &p[n].type,
    &p[n].inH,
    &p[n].inM,
    &p[n].outH,
    &p[n].outM,
    p[n].status,
    &p[n].receiptNo) != EOF)
{
    n++;
}

fclose(fp);

for(i = 0; i < n; i++)
{
    if(strcmp(p[i].vehicleNo, vno) == 0)
    {
        if(strcmp(p[i].status, "OUT") == 0)
        {

```

```

        gotoxy(30,8);
        printf("Vehicle Already Exited!");

        getch();

        return;
    }

    gotoxy(30,7);
    printf("Enter Exit Time (HH MM): ");
    scanf("%d %d",
        &p[i].outH,
        &p[i].outM);

    int fee =
        calculateFee(
            p[i].type,
            p[i].inH,
            p[i].inM,
            p[i].outH,
            p[i].outM);

    if(fee == -1)
    {
        gotoxy(30,9);
        printf("Invalid Exit Time!");

        getch();

        return;
    }

```

```

        strcpy(p[i].status, "OUT");

        dailyIncome += fee;

        gotoxy(30,10);
        printf("Parking Fee: Rs. %d", fee);

        found = 1;

        break;
    }
}

if(!found)
{
    gotoxy(30,10);
    printf("Vehicle Not Found!");

    getch();

    return;
}

fp = fopen("parking.txt", "w");

for(i = 0; i < n; i++)
{
    fprintf(fp,
        "%s %d %d %d %d %d %s %d\n",
        p[i].vehicleNo,
        p[i].type,
        p[i].inH,

```

```

        p[i].inM,
        p[i].outH,
        p[i].outM,
        p[i].status,
        p[i].receiptNo);
    }

    fclose(fp);

    getch();
}

/* ----- VIEW RECORDS ----- */

void viewRecords()
{
    struct Parking p;

    FILE *fp;

    int y = 7;

    system("cls");

    gotoxy(30,4);
    printf("***** PARKING RECORDS *****");

    gotoxy(15,6);
    printf("VehicleNo  Type  Entry  Exit  Status  Receipt");

    fp = fopen("parking.txt", "r");

```

```

if(fp == NULL)
{
    gotoxy(30,10);
    printf("No Records Found!");

    getch();

    return;
}

while(fscanf(fp,
    "%s %d %d %d %d %d %s %d",
    p.vehicleNo,
    &p.type,
    &p.inH,
    &p.inM,
    &p.outH,
    &p.outM,
    p.status,
    &p.receiptNo) != EOF)
{
    gotoxy(15, y);

    printf("%-10s %-5d %02d:%02d %02d:%02d %-5s %-5d",
        p.vehicleNo,
        p.type,
        p.inH,
        p.inM,
        p.outH,
        p.outM,
        p.status,
        p.receiptNo);

```



```

        y++;
    }

    fclose(fp);

    getch();
}

/* ----- SEARCH RECORD ----- */

void searchRecord()
{
    struct Parking p;

    FILE *fp;

    char vno[20];

    int found = 0;

    system("cls");

    gotoxy(30,4);
    printf("***** SEARCH RECORD *****");

    gotoxy(30,6);
    printf("Enter Vehicle Number: ");

    scanf("%s", vno);

    fp = fopen("parking.txt", "r");

```

```

if(fp == NULL)
{
    gotoxy(30,8);
    printf("No Records Found!");

    getch();

    return;
}

while(fscanf(fp,
    "%s %d %d %d %d %d %s %d",
    p.vehicleNo,
    &p.type,
    &p.inH,
    &p.inM,
    &p.outH,
    &p.outM,
    p.status,
    &p.receiptNo) != EOF)
{
    if(strcmp(p.vehicleNo, vno) == 0)
    {
        found = 1;

        gotoxy(30,9);
        printf("Receipt No : %d",
            p.receiptNo);

        gotoxy(30,10);
        printf("Vehicle No : %s",

```

```

        p.vehicleNo);

    gotoxy(30,11);
    printf("Vehicle Type : %d",
        p.type);

    gotoxy(30,12);
    printf("Status : %s",
        p.status);

    break;
}
}

fclose(fp);

if(!found)
{
    gotoxy(30,10);
    printf("Vehicle Record Not Found!");
}

getch();
}

/* ----- DAILY INCOME ----- */

void viewDailyIncome()
{
    system("cls");

    gotoxy(30,6);

```

```

printf("Total Income Today: Rs. %d",
      dailyIncome);

getch();
}

/* ----- ADMIN MENU ----- */

void adminMenu()
{
    int choice;

    do
    {
        system("cls");

        gotoxy(30,4);
        printf("***** ADMIN MENU *****");

        gotoxy(30,6);
        printf("1. Vehicle Entry");

        gotoxy(30,7);
        printf("2. Vehicle Exit");

        gotoxy(30,8);
        printf("3. View Records");

        gotoxy(30,9);
        printf("4. Search Record");

        gotoxy(30,10);

```

```
printf("5. View Daily Income");
```

```
gotoxy(30,11);
```

```
printf("6. Logout");
```

```
gotoxy(30,13);
```

```
printf("Enter Choice: ");
```

```
scanf("%d", &choice);
```

```
switch(choice)
```

```
{
```

```
    case 1:
```

```
        vehicleEntry();
```

```
        break;
```

```
    case 2:
```

```
        vehicleExit();
```

```
        break;
```

```
    case 3:
```

```
        viewRecords();
```

```
        break;
```

```
    case 4:
```

```
        searchRecord();
```

```
        break;
```

```
    case 5:
```

```
        viewDailyIncome();
```

```
        break;
```

```

        case 6:
            break;

        default:
            gotoxy(30,15);
            printf("Invalid Choice!");

            getch();
        }

    }
    while(choice != 6);
}

/* ----- USER MENU ----- */

void userMenu()
{
    int choice;

    do
    {
        system("cls");

        gotoxy(30,4);
        printf("***** USER MENU *****");

        gotoxy(30,6);
        printf("1. Vehicle Entry");

        gotoxy(30,7);
        printf("2. Vehicle Exit");
    }

```

```
gotoxy(30,8);
printf("3. Search Record");

gotoxy(30,9);
printf("4. Logout");

gotoxy(30,11);
printf("Enter Choice: ");

scanf("%d", &choice);

switch(choice)
{
    case 1:
        vehicleEntry();
        break;

    case 2:
        vehicleExit();
        break;

    case 3:
        searchRecord();
        break;

    case 4:
        break;

    default:
        gotoxy(30,13);
        printf("Invalid Choice!");
```

```

        getch();
    }

}

while(choice != 4);
}

/* ----- MAIN FUNCTION ----- */

int main()
{
    int choice;

    int loggedIn = 0;

    while(!loggedIn)
    {
        system("cls");

        gotoxy(25,4);
        printf("***** WELCOME TO PARKING SYSTEM *****");

        gotoxy(30,6);
        printf("1. Register");

        gotoxy(30,7);
        printf("2. Login");

        gotoxy(30,8);
        printf("3. Exit");
    }
}

```



```

gotoxy(30,10);
printf("Enter Choice: ");

scanf("%d", &choice);

if(choice == 1)
{
    registerUser();
}

else if(choice == 2)
{
    loggedIn = loginUser();

    if(loggedIn)
    {
        if(strcmp(currentRole, "admin") == 0)
        {
            adminMenu();
        }

        else
        {
            userMenu();
        }

        loggedIn = 0;
    }
}

else if(choice == 3)
{

```

```
    system("cls");

    gotoxy(25,10);
    printf("THANK YOU FOR USING SYSTEM");

    getch();

    exit(0);
}

else
{
    gotoxy(30,12);
    printf("Invalid Choice!");

    getch();
}
}

return 0;
}
```

Chapter 12 : System Architecture

12.1. Data Flow Diagram

12.1.1. DFD Diagram Level 0

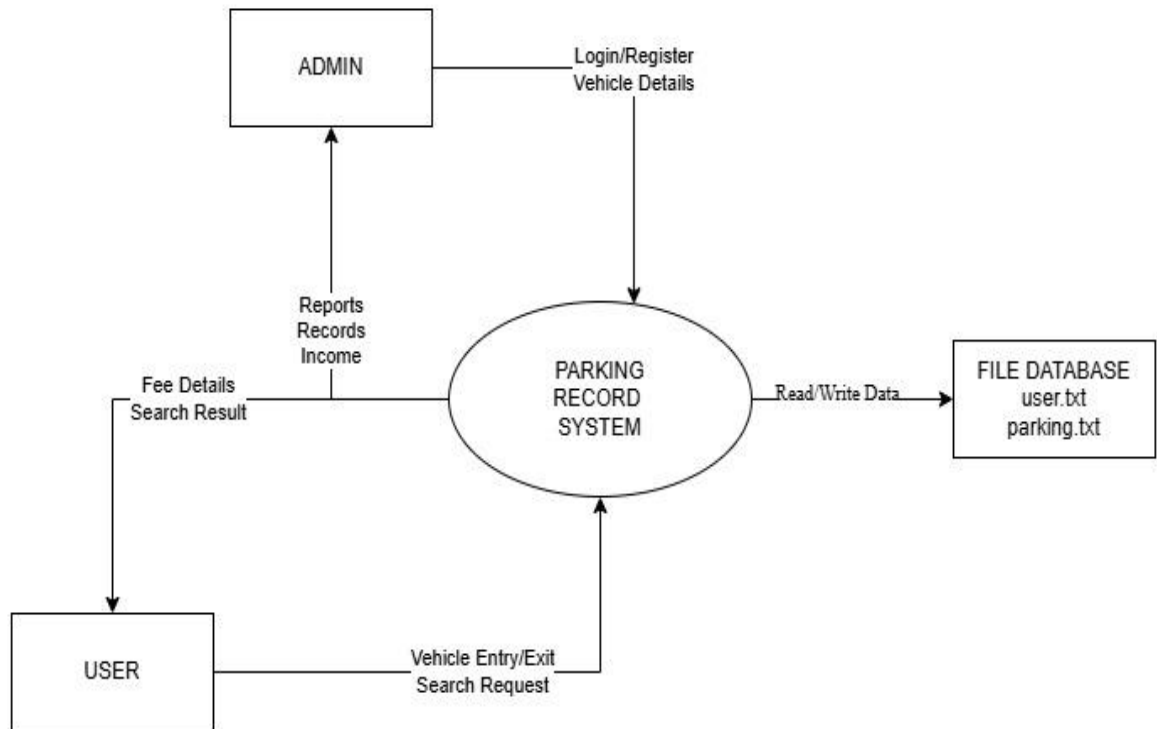


Figure 3: DFD Level 0

12.1.2. DFD Diagram Level 1

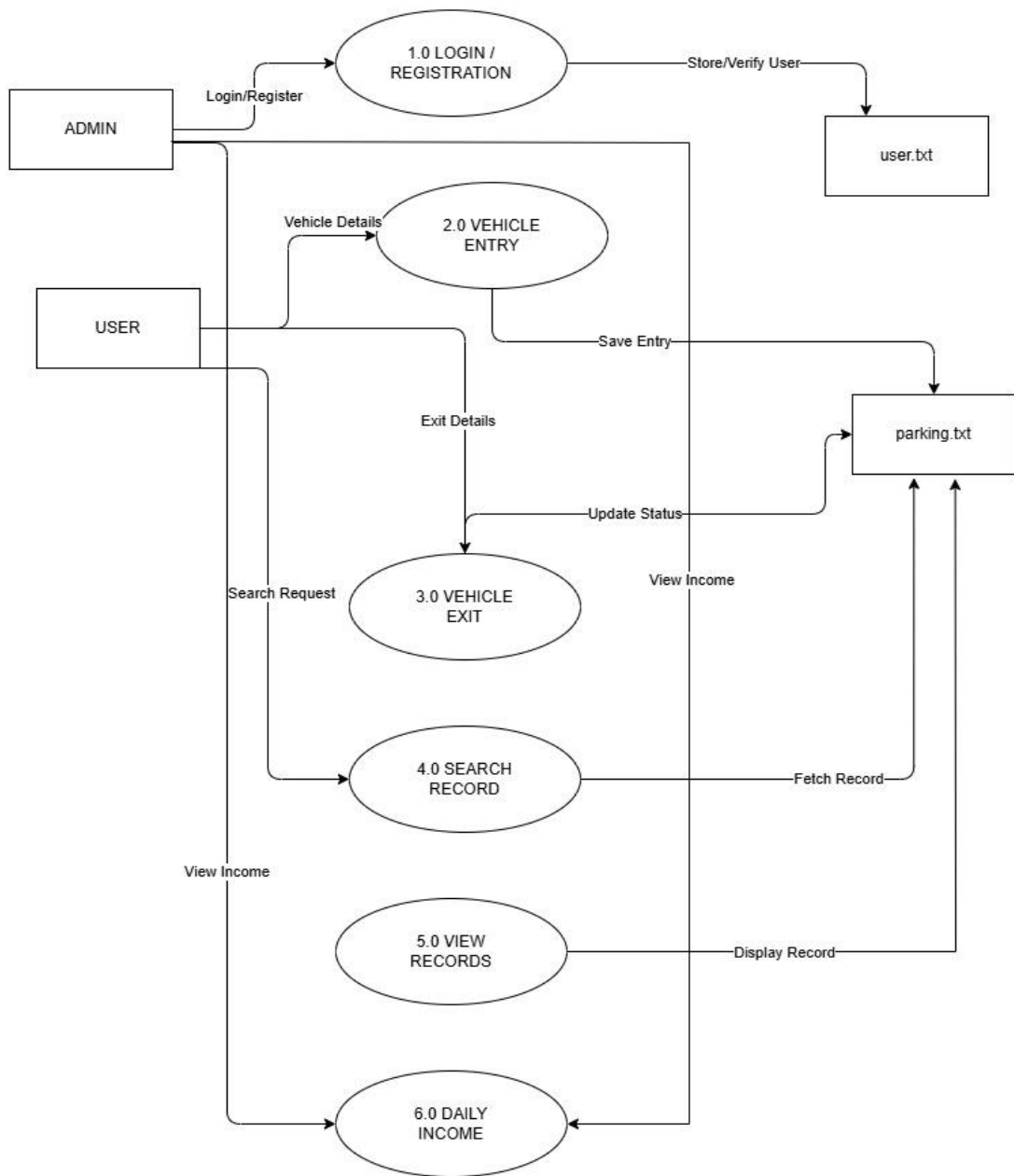


Figure 4: DFD Level 1

12.2. Object Oriented

12.2.1. Use case diagram

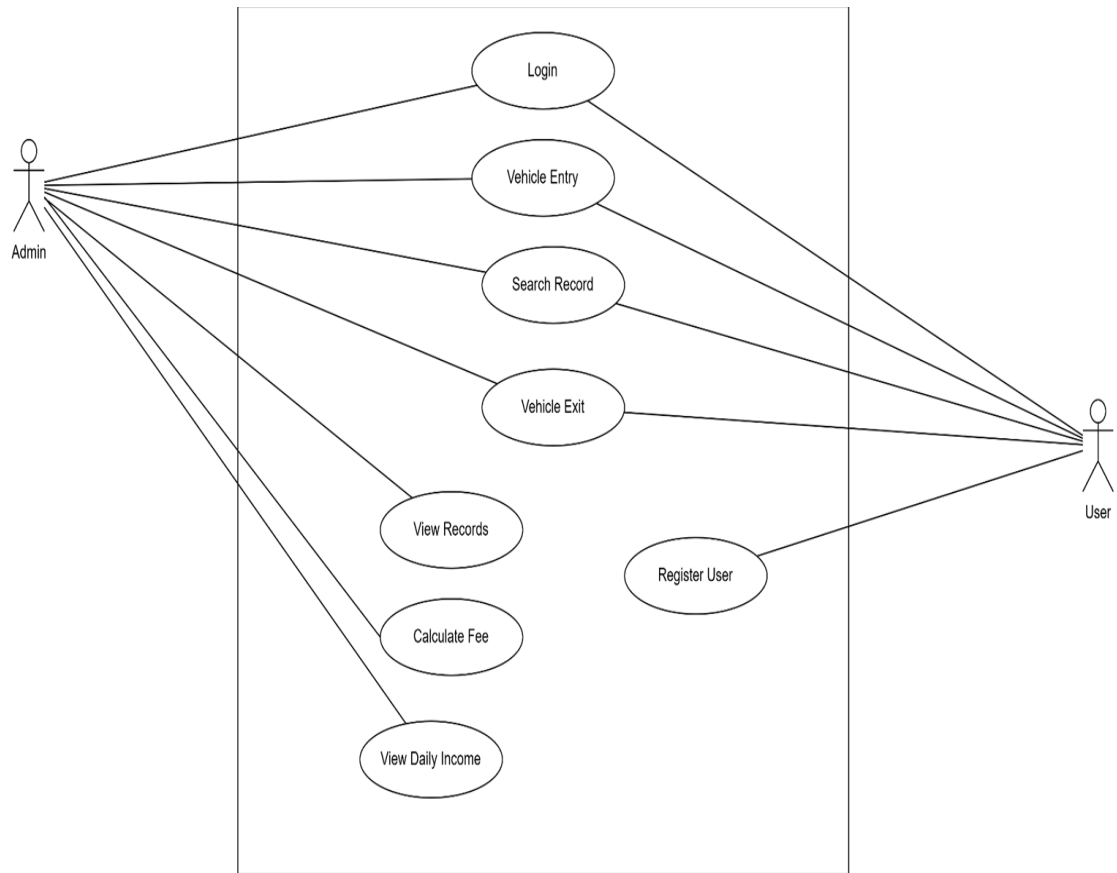


Figure 5: Use Case

12.2.2. Sequence Diagram

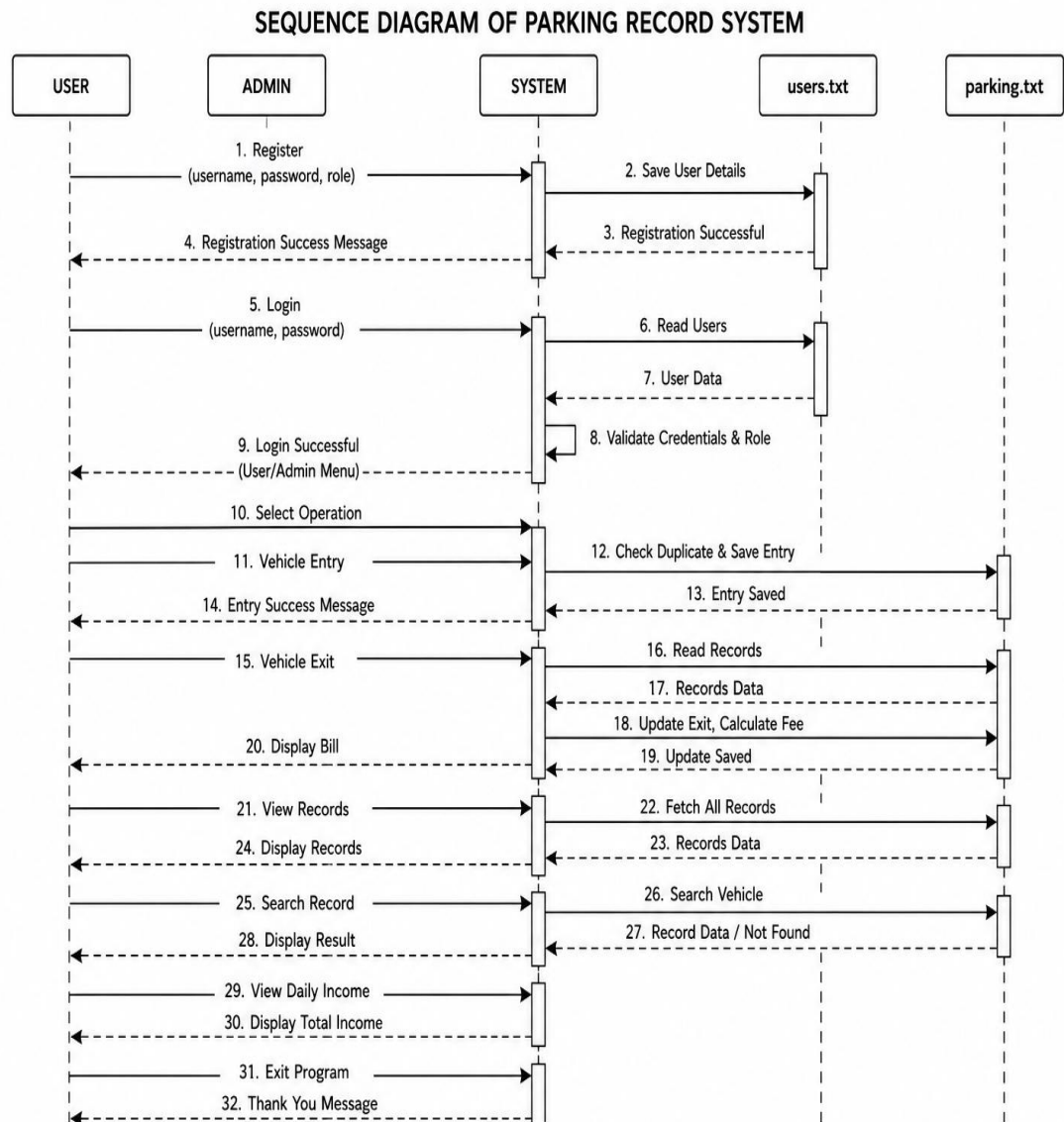


Figure 6: Sequence Diagram

Chapter 13

13. SYSTEM DEVELOPMENT AND IMPLEMENTATION

13.1 System Development

The Parking Record System is developed using the C programming language to manage parking activities efficiently. The system is designed to maintain vehicle registration, entry records, exit records, parking fee calculation, parking slot management, and total income reports.

The development of the system follows a structured approach to ensure proper functionality and easy maintenance. Different modules are created for each operation such as vehicle registration, vehicle entry, vehicle exit, fee calculation, file handling, and report generation.

The system uses file handling concepts for permanent storage of data. Files such as user.txt and parking.txt are used to store vehicle details and parking records. This allows records to remain saved even after the program is closed.

The major development activities include:

- Requirement Analysis
- System Design
- Coding
- Testing
- Debugging
- Implementation

The program is developed in a simple and user-friendly manner so that users can easily operate the parking system without difficulty.

13.2 System Implementation

The implementation of the Parking Record Keeping System is done using the Dev C/C++ compiler in the Windows operating system. The program uses file handling to store and retrieve parking records from files such as user.txt and parking.txt.

During implementation:

- Vehicles are first registered in the system.
- Only registered vehicles are allowed to enter the parking area.
- Entry and exit times are recorded automatically.
- Parking fees are calculated based on vehicle type and parking duration.
- Parking slot availability is checked before vehicle entry.
- Reports of total income and exited vehicles are generated.
- All records are updated and stored permanently in files.

The system is tested with different vehicle records to ensure proper functioning of registration, entry, exit, fee calculation, and slot management features.

13.3. Programming platform

The Parking Record Keeping System is developed using the C programming language on a Windows-based programming platform. The system is a console-based application that manages vehicle parking records, calculates parking fees, and stores data permanently using text files.

The platform was selected because it is simple, lightweight, easy to understand, and suitable for academic mini-project development.

13.4. Operating Environment

The operating environment refers to the hardware and software environment required to develop, execute, and maintain the Parking Record Keeping System. The system is developed as a console-based application using the C programming language and operates in a Windows environment.

Chapter 14

14. Testing and Debugging

14.1 Tools Used in Testing

S.N	Tool	Specification
1.	Code::Blocks	Version 20.03 or above
2.	GCC Compiler	GCC 9.2.0 or above
3.	Windows 10	64-bit Operating System
4.	Command Prompt	Default Window Console
5.	Notepad++	Version 8.0 or above
6.	Draw.io	Web/Desktop Version
7.	StarUML	Version 5.0 or above
8.	Text File System	.txt File format

14.2 Test Case

Registration and Login Case

ID	Description	Input Data	Expected result	Actual result	Status
T-01	Create new user	user1/pass123	Account created successfully	Account created successfully	PASS
T-02	Blank input check	-/-	Show validation	Validation error shown	PASS
T-03	Weak password check	user1/123	Reject weak password	Weak password	FAIL
T-04	Correct credentials	user1/pass123	Login successful	Login successful	PASS
T-05	Wrong password	user1/wrong	Login failed	Login failed	PASS
T-06	Non-existing user	userX/pas123	Login failed	System allowed login	FAIL
T-07	No input given	-/-	Show error message	Error message displayed	PASS

Entry and Exit

ID	Description	Input data	Expected result	Actual result	Status
T-01	Bike entry	2W/BA-1234	Entry recorded	Entry recorded	PASS
T-	Car entry	4W/BA-	Entry	Entry	PASS

02		5678	recorded	recorded	
T-03	Wrong type input	3W/BA-9999	Reject input	Input accepted incorrectly	FAIL
T-04	No number provided	2W/-	Reject entry	Entry rejected	PASS
T-05	Existing vehicle exit	BA-1234	Exit recorded	Exit recorded	PASS
T-06	Vehicle not found	BA-9999	Show error	No error shown	FAIL
T-07	Repeat exit attempt	BA-1234 AGAIN	Prevent duplication	Duplication exit allowed	FAIL
T-08	No entry record	BA-7777	Reject request	Request rejected	PASS

Report Test Case

ID	Description	Input data	Expected result	Actual result	Status
T-01	Display Record	View Command	Show Vehicle list	Vehicle list shown	Pass
T-02	Total earning	Report request	Correct income displayed	Incorrect income shown	Fail
T-03	Slot availability	Dashboard	Correct slot count	Correct slot count	Pass
T-04	No record case	Empty DB	Show "No records	Message Display	Pass

			found”		
--	--	--	--------	--	--

15.Conclusion

The Parking Record Keeping System was successfully developed to efficiently manage the entry, exit, and tracking of vehicles in a parking area. The system simplifies the process of recording two-wheelers and four-wheelers, calculates total income, and maintains accurate vehicle records. It reduces manual effort and minimizes errors in data management, making the parking process more organized and reliable.

This project demonstrates how a structured C program can be used to solve real-world problems such as parking management. By integrating features like user authentication, vehicle tracking, and income reporting, the system improves operational efficiency. Although it has certain limitations, such as a console-based interface and fixed slot capacity, it provides a strong foundation for further enhancements like database integration and graphical user interface development.